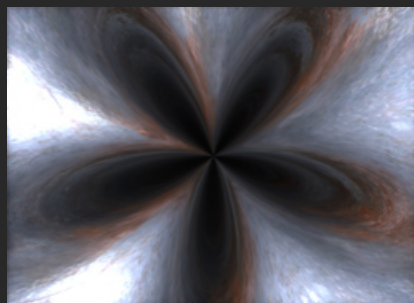
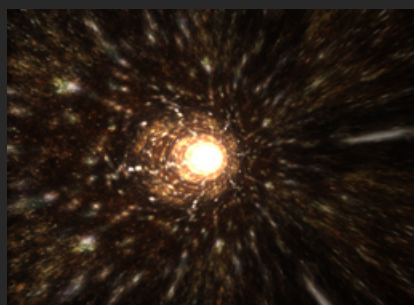
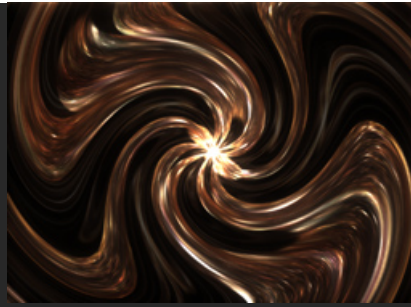
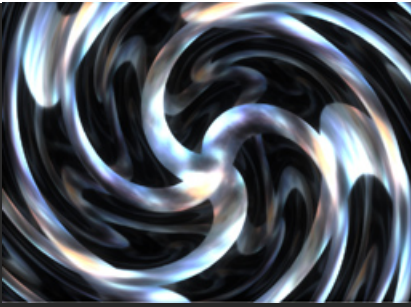


Inigo Quilez :: articles :: plane deformations - 1999



One of the nicest oldschool effects are the 2D LUT deformations and tunnels. These were very fast to compute in realtime, in software (without GPUs), and very spectacular too, so they were very common back in the late 1990s when I first learnt about them. The idea is to deform a texture by some mathematical function. But because evaluating mathematical functions for every pixel in the screen was impossible back then, a texture was used instead as a LUT (Look Up Table). That way it was possible to create animated flowers, fake 3d landscapes, tunnels, whirlpools, holes, etc. As I said, I used them a lot during 1999 and 2000 myself, you can see them in action in demos like [Rare](#) or [Storm](#).





Today you can do the same effect very easily by using pixel shaders, and with much less effort at the same quality, since the GPU will do lot of mipmapping and bilinear filtering, that otherwise you have to do manually.

In either case the idea is to take the screen buffer, and map a texture to it. For example, a trivial way to display such a texture would be to do something like this:

```
void renderDeformation( uint32_t *buffer, float time )
{
    for( uint32_t j=0; j<mYres; j++ )
        for( uint32_t i=0; i<mXres; i++ )
        {
            buffer[mXres*j+i] = mTexture[ 256*(j&255) + (i&255) ];
        }
}
```

where I assume the texture is of 256 x 256 pixels, and the screen is mXres x mYres pixels. The texture will repeat over the screen as needed, thank to the AND operations. This code just copies one texel to one pixel. The same code in a pixel shader (GLSL) would look like this:

```
uniform sampler2D mTexture;
void main( void )
{
    gl_FragColor = texture( mTexture, gl_TexCoord[0].xy );
}
```

Now that we have our texture displayed on screen we can start coding the real effect. The trick is to modify the texture coordinates (i,j) in the software rendering version or gl_TexCoord[0].xy in the GLSL version with crazy formulas to get nice deformations. Instead of adding lots of calculations to our code, we will precompute the formulas in some look up table (LUT). That means that we will input the coordinates to the LUT and the LUT will give us some new coordinates back. Something like this:

```
void renderDeformation( uint32_t *buffer, float time )
{
    const uint32_t itime = (uint32_t)(10.0f*time);
    for( uint32_t j=0; j<mYres; j++ )
```

```

for( uint32_t i=0; i<mXres; i++ )
{
    const uint32_t o = xres*j + i;
    const uint8_t u = mLUT[ 2*o+0 ] + itime;
    const uint8_t v = mLUT[ 2*o+1 ] + itime;
    buffer[mXres*j+i] = mTexture[ 256*(v&255) + (u&255) ];
}
}

```

Obviously this can be optimized to the following:

```

void renderDeformation( int *buffer, float time )
{
    const uint32_t itime = (uint32_t)(10.0f*time);
    const uint32_t numPixels = mXres*mYres;
    for( uint32_t i=0; i<numPixels; i++ )
    {
        buffer[i] = mTexture[ (mLut[i]+itime) & 0xffffu ];
    }
}

```

In the case of GLSL, this would look like this:

```

uniform float      time;
uniform sampler2D mTexture;
uniform sampler2D mLUT;
void main( void )
{
    vec4 uv = texture( mLUT, gl_TexCoord[0].xy );
    vec4 co = texture( mTexture, uv.xy + vec2(time) );
    gl_FragColor = co;
}

```

Adding a time factor to the texture coordinates is done so that the texture moves. It's now up to us to be imaginative and fill the LUT with interesting things. For example,

```

void createLUT( void )
{
    uint32_t k = 0;
    for( int j=0; j<mYres; j++ )
    for( int i=0; i<mXres; i++ )
    {
        const float x = -1.0f + 2.0f*(float)i/(float)mXres;
        const float y = -1.0f + 2.0f*(float)j/(float)mYres;
        const float d = sqrtf( x*x + y*y );
        const float a = atan2f( y, x );
    }
}

```

```

    // magic formulas here
    const float u = cosf( a )/d;
    const float v = sinf( a )/d;

    mLUT[k++] = ((int)(256.0f*u)) & 255;
    mLUT[k++] = ((int)(256.0f*v)) & 255;
}
}

```

will produce an interesting deformation for the software rendering version. As shown in the code the LUT stores 16 bit integer values per channel. If we were implementing the effect in GLSL, we better use a floating point texture for the LUT (float16 is more than enough, we don't need a full 32 bit floating point precision), and the code to fill the texture would be

```

void createLUT( float *lutTexture )
{
    for( uint32_t j=0; j<mYres; j++ )
        for( uint32_t i=0; i<mXres; i++ )
        {
            const float x = -1.0f + 2.0f*(float)i/(float)mXres;
            const float y = -1.0f + 2.0f*(float)j/(float)mYres;
            const float d = sqrtf( x*x + y*y );
            const float a = atan2f( y, x );

            // magic formulas here
            const float u = cosf( a )/d;
            const float v = sinf( a )/d;

            *lutTexture++ = fmodf( u, 1.0f );
            *lutTexture++ = fmodf( v, 1.0f );
        }
}

```

It's lots of fun to invent as many formulas as you can. All the images on this article were made with this simple algorithm, by using the following formulas:

$u = x \cdot \cos(2 \cdot r) - y \cdot \sin(2 \cdot r)$	$u = 0.3 / (r + 0.5 \cdot x)$	$u = 0.02 \cdot y + 0.03 \cdot \cos(a \cdot 3) / r$	$u = 0.1 \cdot x / (0.11 + r \cdot 0.5)$
$v = y \cdot \cos(2 \cdot r) + x \cdot \sin(2 \cdot r)$	$v = 3 \cdot a / \pi$	$v = 0.02 \cdot x + 0.03 \cdot \sin(a \cdot 3) / r$	$v = 0.1 \cdot y / (0.11 + r \cdot 0.5)$
$u = 0.5 \cdot a / \pi$	$u = r \cdot \cos(a + r)$	$u = 1 / (r + 0.5 + 0.5 \cdot \sin(5 \cdot a))$	$u = x / \text{abs}(y)$
$v = \sin(7 \cdot r)$	$v = r \cdot \sin(a + r)$	$v = a \cdot 3 / \pi$	$v = 1 / \text{abs}(y)$

Note that GPU shaders are very powerful these days, and you don't actually need LUTs anymore, you can do

all the calculations per pixel and include the time factor really inside the formulas. In fact, you can see some of the formulas above implemented as realtime shaders here:

<https://www.shadertoy.com/view/Ms2SWW>

<https://www.shadertoy.com/view/Xsf3Rn>

<https://www.shadertoy.com/view/XdXGzn>

<https://www.shadertoy.com/view/Xdf3Rn>

<https://www.shadertoy.com/view/4sXGzn>

<https://www.shadertoy.com/view/4sXGRn>

inigo quilez - learning computer graphics since 1994